Ingeniería de Aplicaciones Web

Diego C. Martínez

Departamento de Ciencias e Ingeniería de la Computación
Universidad Nacional del Sur

Rendering

NEXT.JS

Client Side Rendering

La carga del procesamiento necesario para generar la página recae en el cliente

Server Side Rendering

La carga del procesamiento necesario para generar la página recae en el servidor

En la práctica nunca es 100% uno u otro

ANALOGÍA

En CSR el cliente "arma el mueble en casa", con algunas partes obtenidas del servidor

En SSR el servidor "se lo entrega armado" al cliente

Rendering

Client Side Rendering

Usarlos cuando realmente corresponda

Server Side Rendering

Server Components

NO PUEDEN

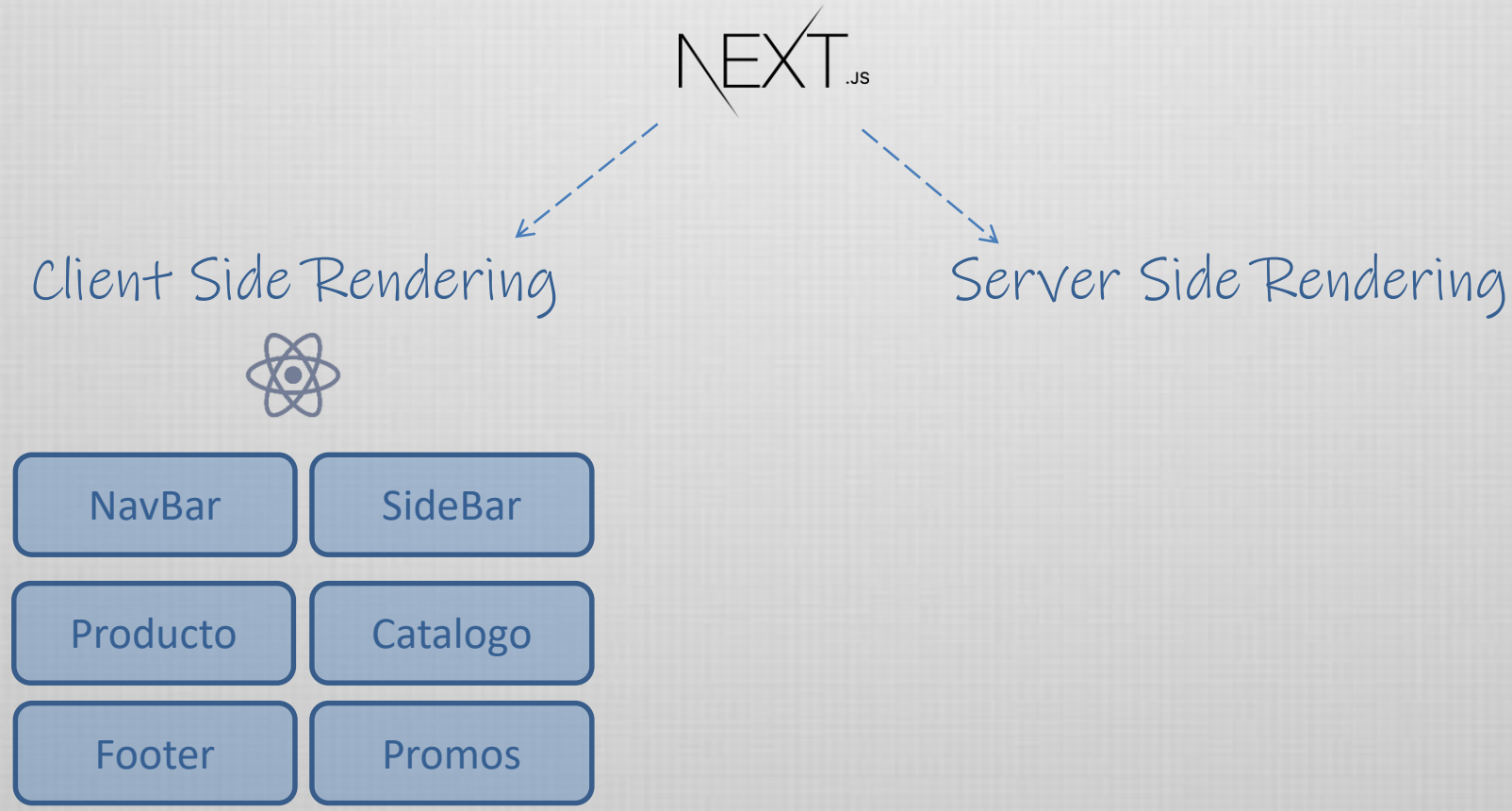
Escuchar eventos del browser

Acceder a APIs del browser

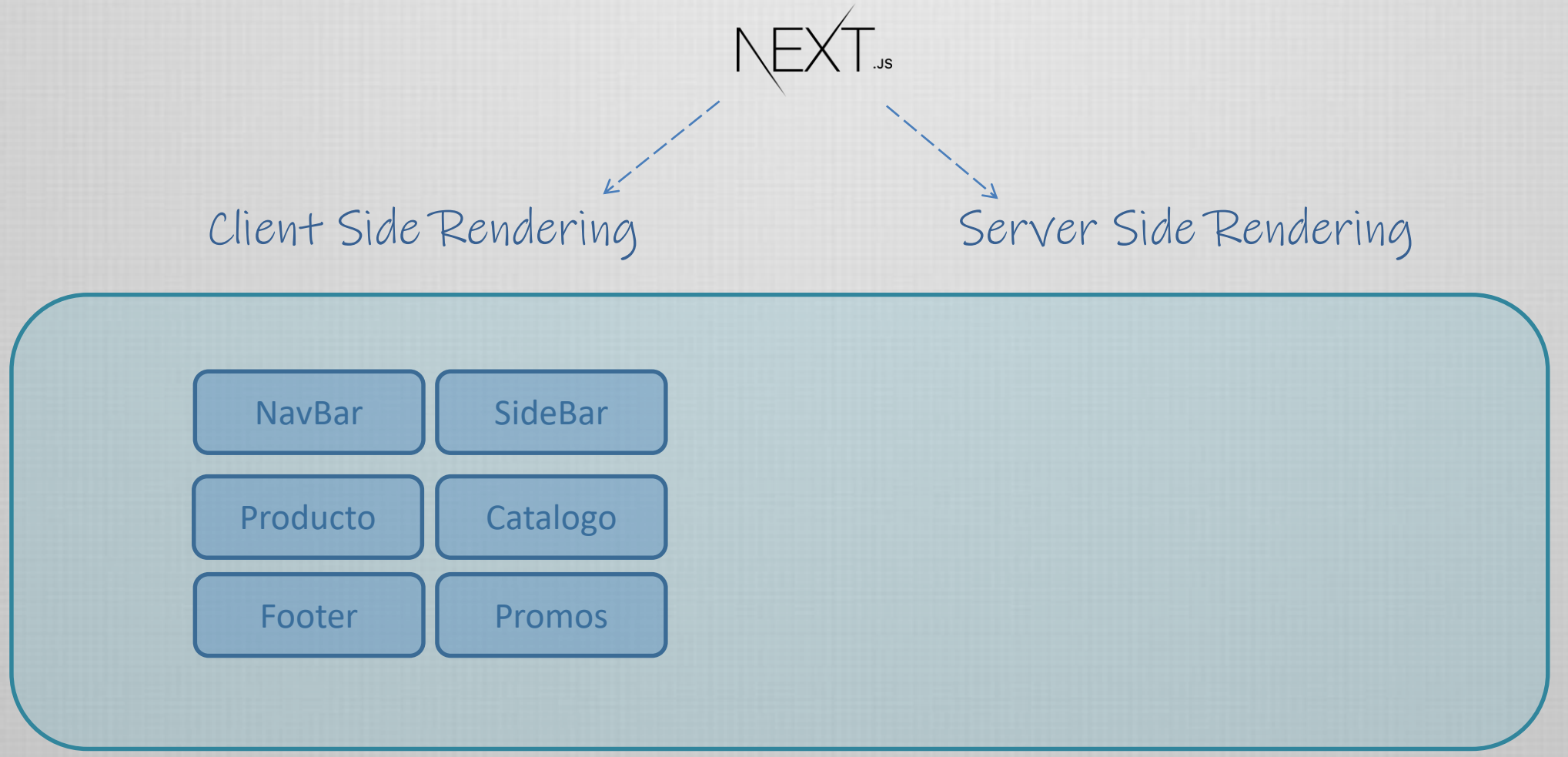
Mantener el estado local interactivo

Usar effects

Rendering


NEXT.JS
Client Side Rendering
Server Side Rendering
NavBar
SideBar
Producto
Catalogo
Footer
Promos

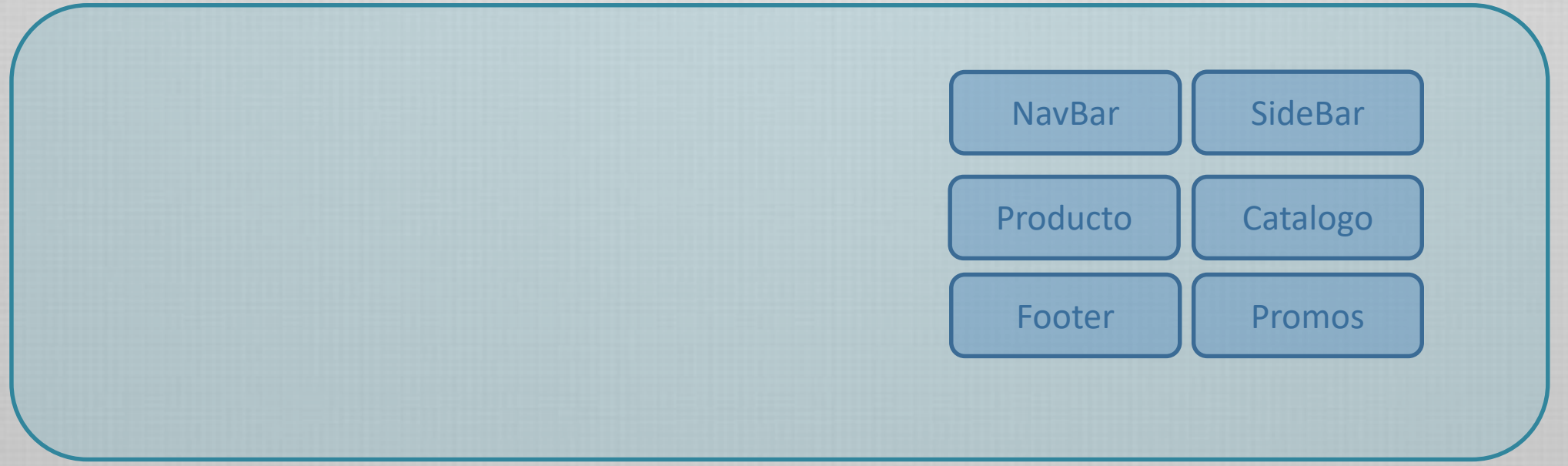
Rendering


NEXT.JS
Client Side Rendering
Server Side Rendering
NavBar
SideBar
Producto
Catalogo
Footer
Promos
Aplicación Web

Rendering

Client Side Rendering

Server Side Rendering

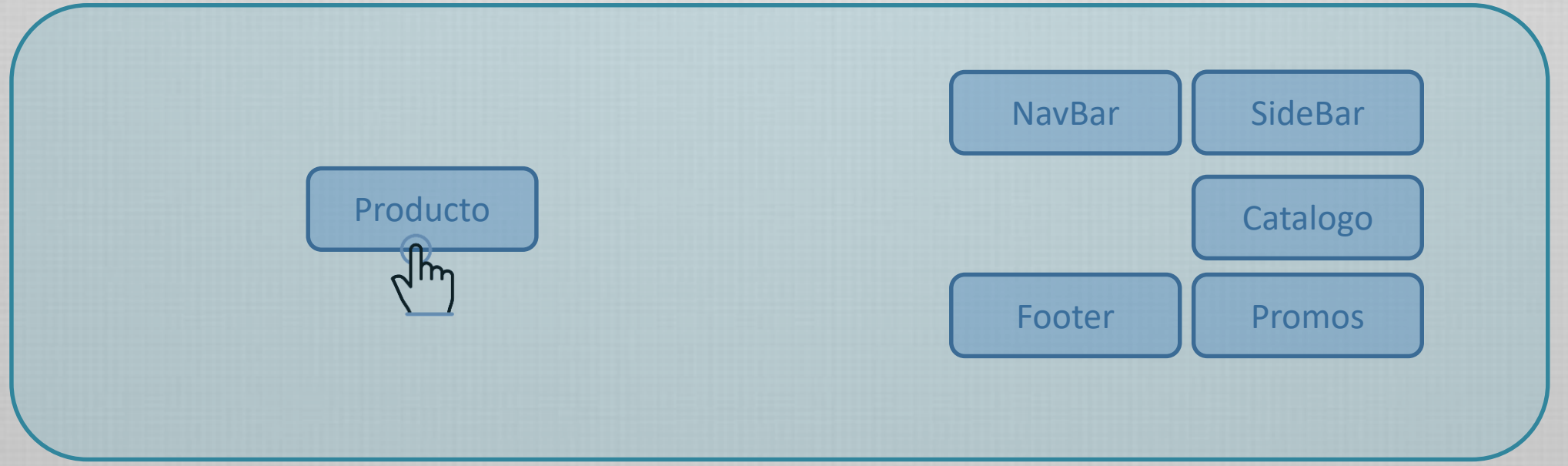


Aplicación Web

Rendering

Client Side Rendering

Server Side Rendering

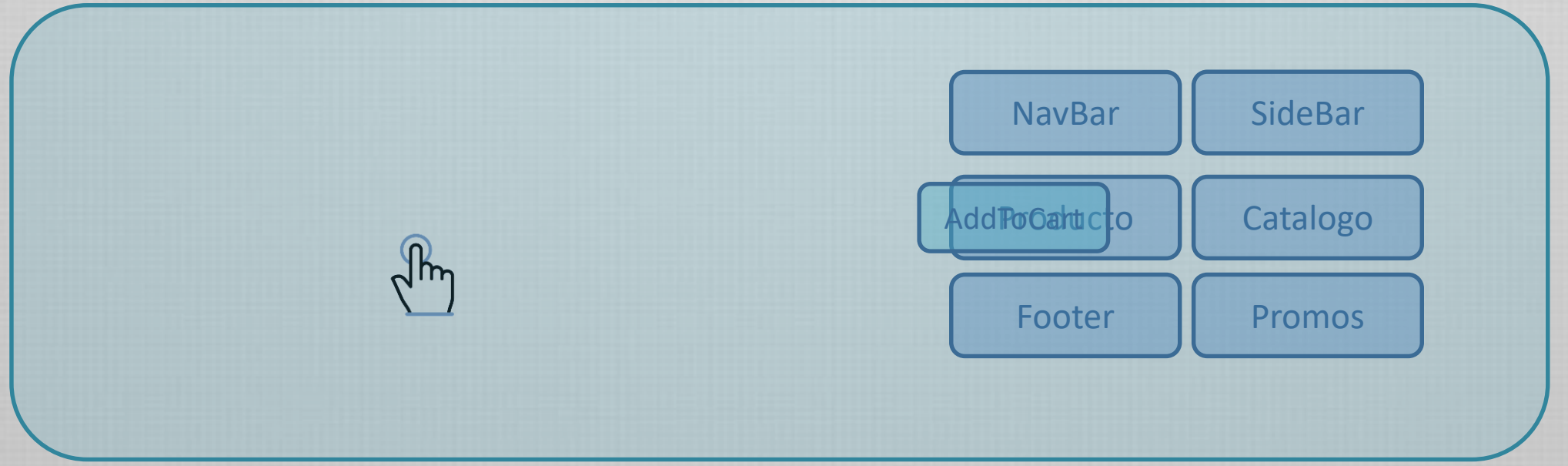


Aplicación Web

Rendering

Client Side Rendering

Server Side Rendering



Aplicación Web

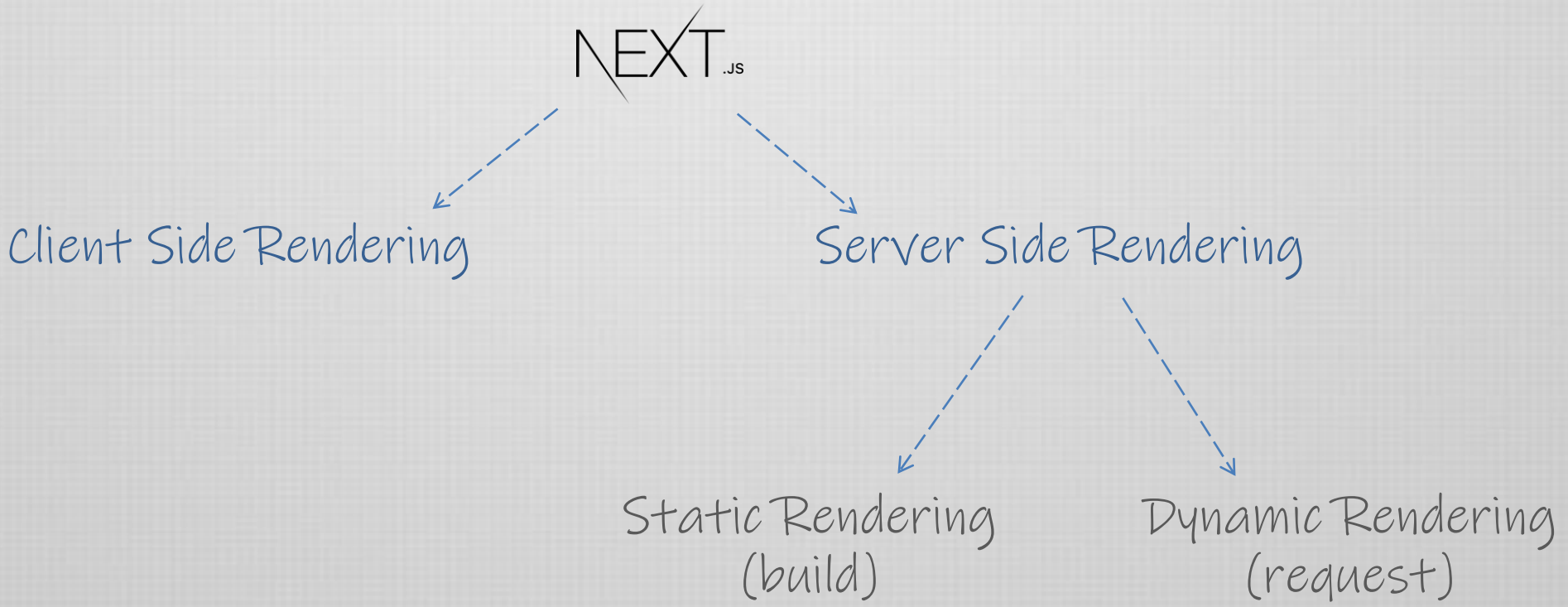
Rendering

NEXT.JS

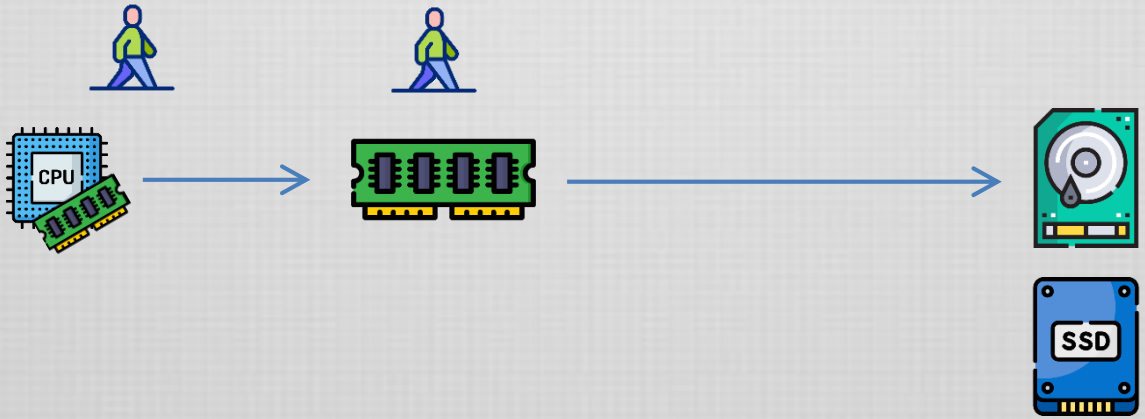
Client Side Rendering

Server Side Rendering

Rendering

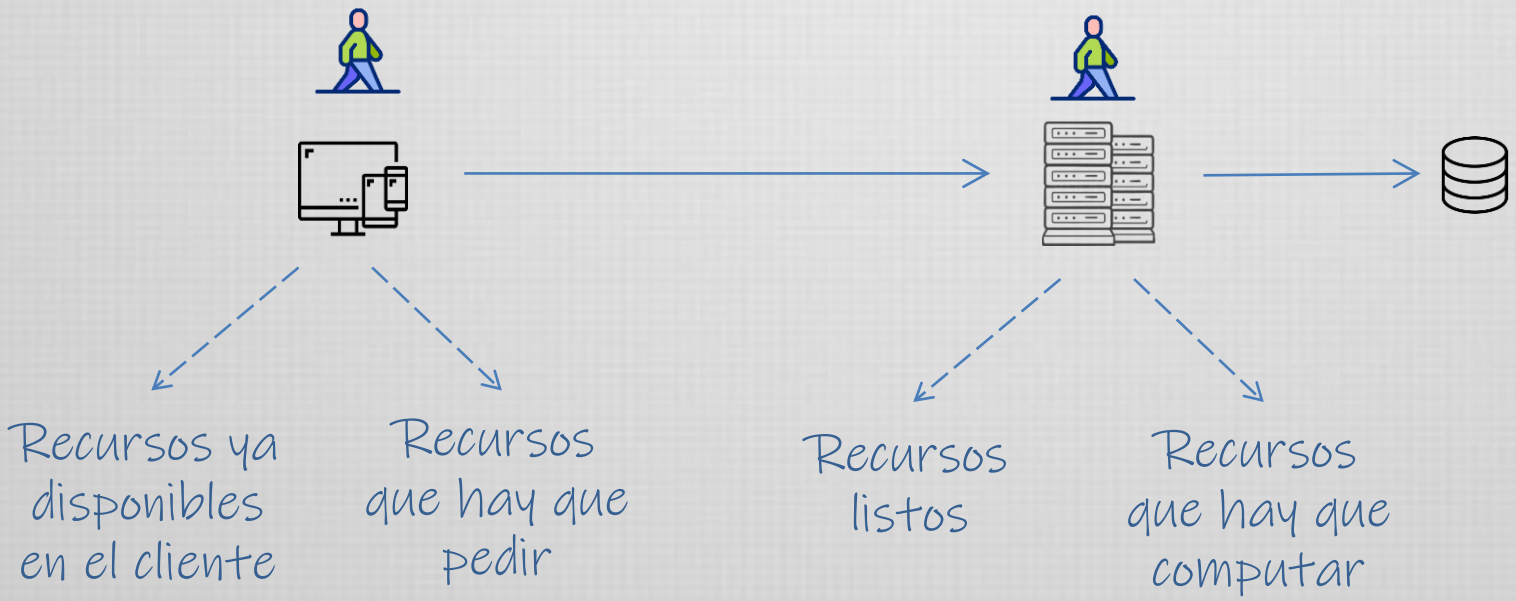

NEXT.JS
Client Side Rendering
Server Side Rendering
Static Rendering
(build)
Dynamic Rendering
(request)

Eficiencia y cercanía



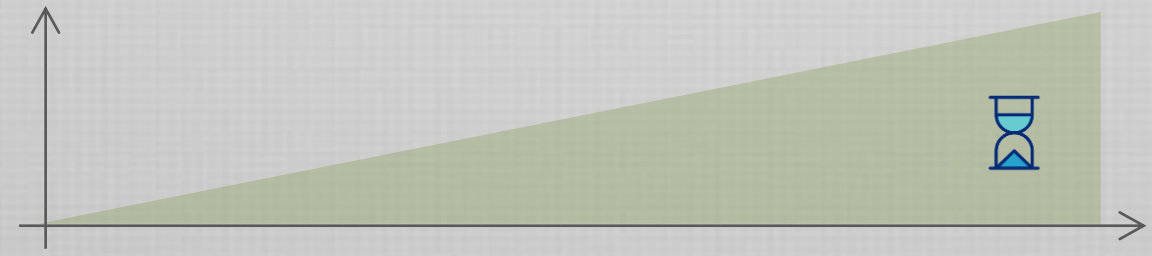
La mejor forma de reducir la latencia
es tener los datos requeridos
lo más cerca posible del usuario

Eficiencia y cercanía

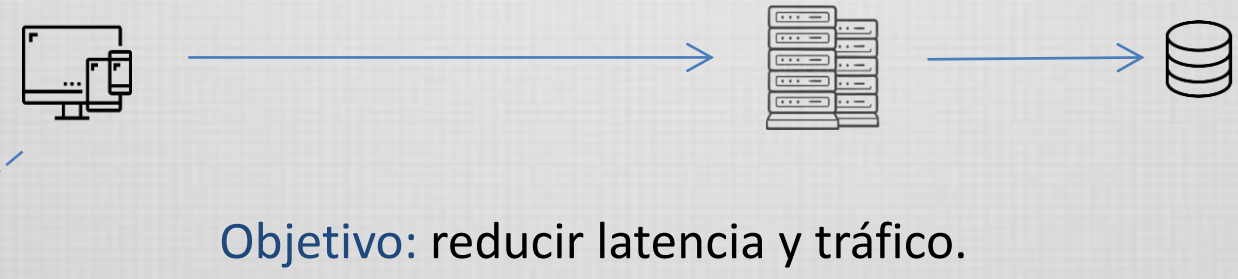


Si el dato no cambia, ¿para qué vamos a computarlo cada vez que alguien lo pide?

Lo mejor sería dejarlo ya 'cocinado' en la puerta del navegador



Eficiencia y cercanía



Recursos ya disponibles en el cliente

caché del navegador

Objetivo: reducir latencia y tráfico.

Beneficios directos en:

- tiempo de carga
- consumo de red
- experiencia de usuario

Es usualmente SQLite o archivos indexados

Recursos más "cacheados": *JS, CSS, HTML, imágenes*

Eficiencia y cercanía

Recursos ya disponibles en el cliente

caché del navegador

¿Cuándo un recurso "cacheado" es aún válido?

Dos estrategias fundamentales:

Expiración

El recurso se considera válido sin preguntar al servidor

```
Cache-Control: max-age=3600
```

```
Expires: Wed, 21 Oct 2026 07:28:00 GMT
```

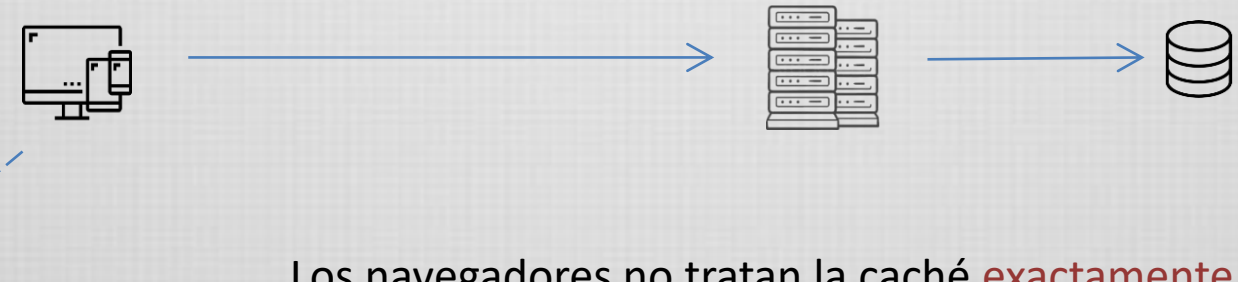
Revalidación

El navegador pregunta si el recurso cambió

```
Etag: "abc123"
```

```
If-None-Match: "abc123"
```

Eficiencia y cercanía



Recursos ya disponibles en el cliente

caché del navegador

Los navegadores no tratan la caché exactamente de la misma forma

En general, respetan los encabezados HTTP que controlan la cache, como

```
Cache-ControlExpiresETag
If-None-MatchLast-Modified
If-Modified-Since
```

Algunos navegadores almacenan más tiempo los recursos aunque no se haya especificado explícitamente una política

No todos asignan el mismo espacio local

Eficiencia y cercanía

Recursos ya disponibles en el cliente

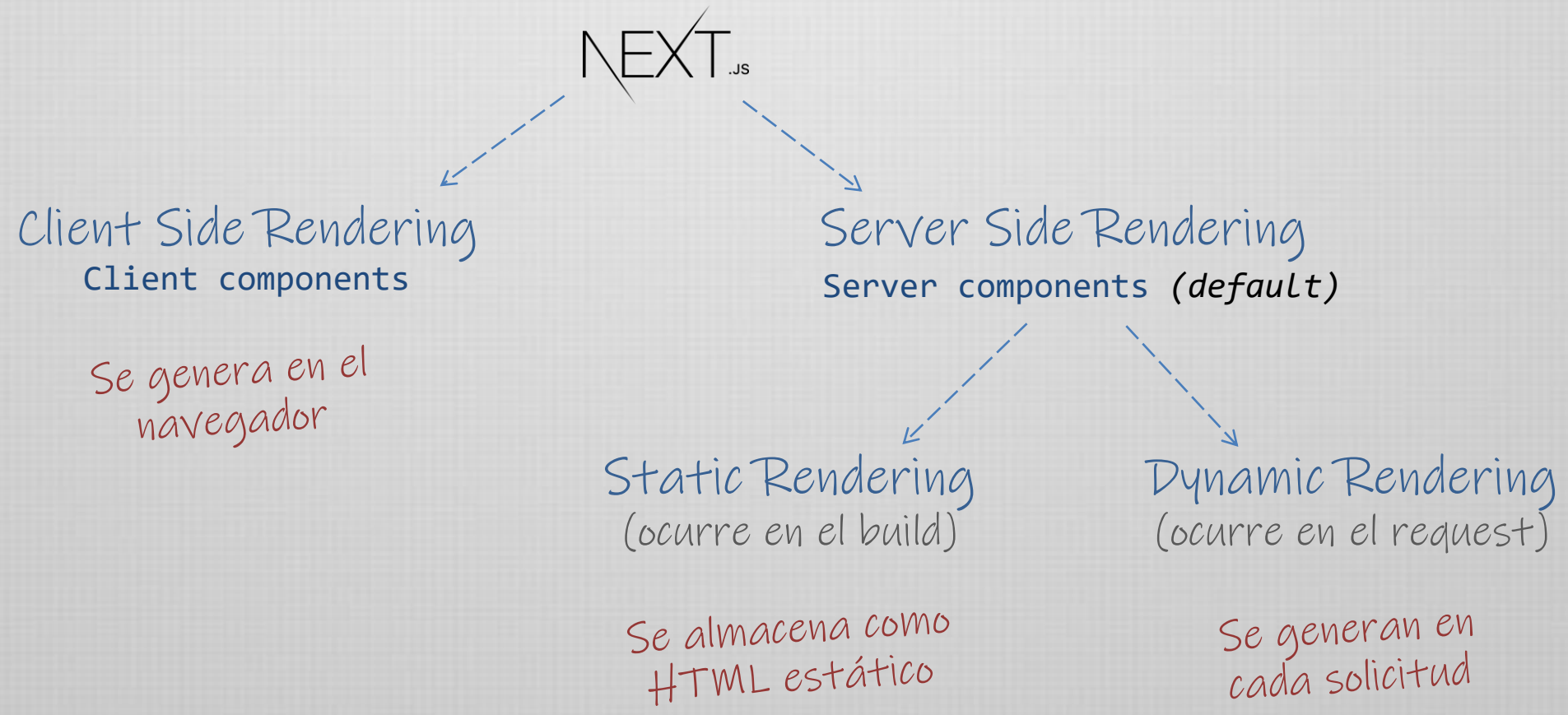
El resto de los recursos hay que pedirlos

caché del navegador

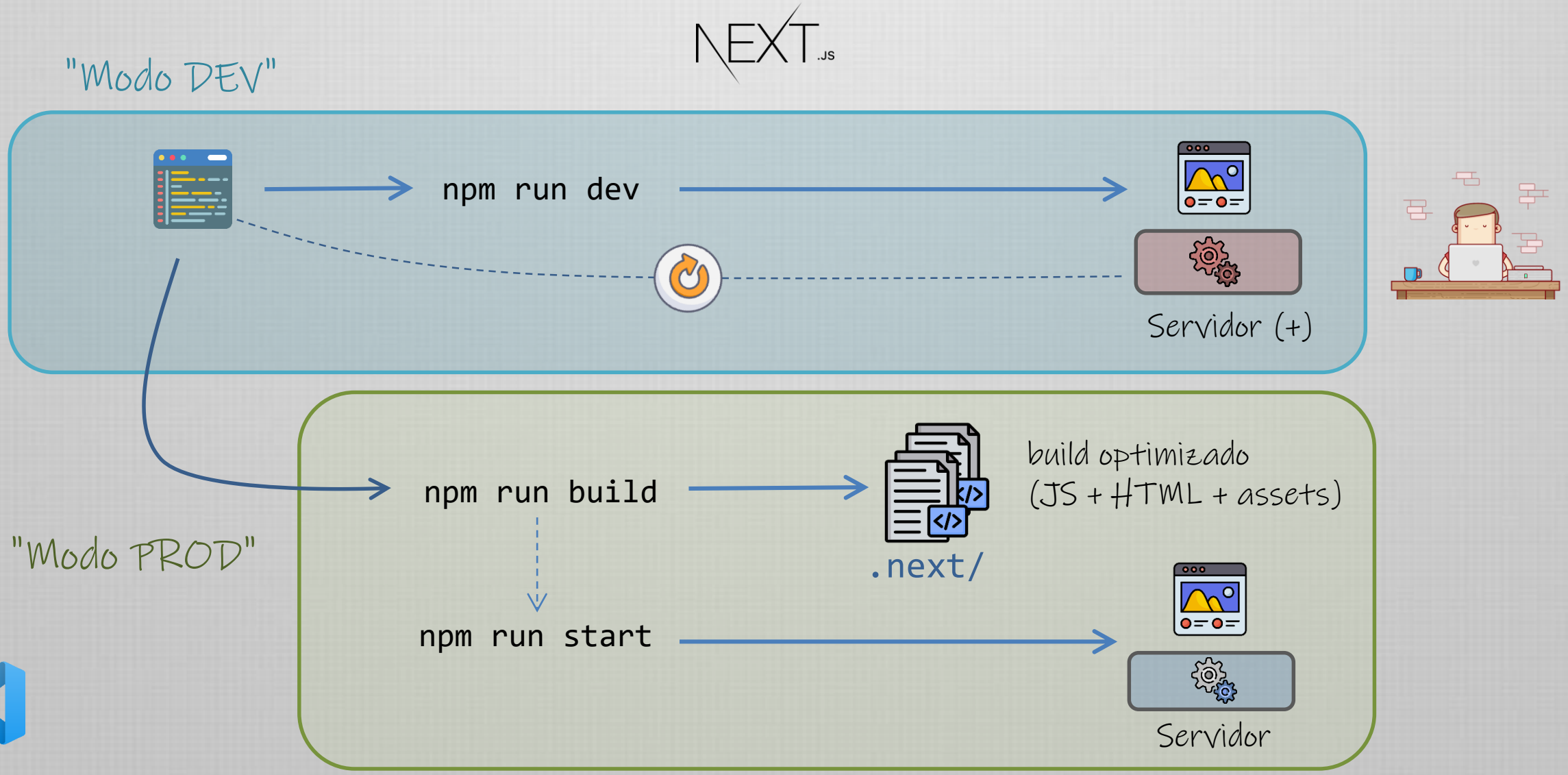
Lo más rápido para el usuario

Cuanto más podamos "cocinar" con anticipación, MEJOR

Rendering


NEXT.JS
Client Side Rendering
Client components
Se genera en el navegador
Server Side Rendering
Server components (default)
Static Rendering
(ocurre en el build)
Se almacena como HTML estático
Dynamic Rendering
(ocurre en el request)
Se generan en cada solicitud

Rendering


NEXT.JS
"Modo DEV"
npm run dev
Servidor (+)
"Modo PROD"
npm run build
.next/
build optimizado
(JS + HTML + assets)
npm run start
Servidor

Rendering

Server Side Rendering

`Server components` *(default)*

Static Rendering
(build)

+ Máximo rendimiento

+ Ideal para contenido que no cambia con frecuencia

- Si los datos cambian, es necesario un nuevo *build*

La tendencia preferida, al menos que se hagan indicaciones explícitas

Dynamic Rendering
(request)

Ideal para contenido altamente dinámico que depende de datos en tiempo real

Inevitable para muchas funcionalidades

Rendering

Server Side Rendering

Server components *(default)*

Static Rendering (build)

```
fetch(url);
```

Dynamic Rendering (request)

```
fetch(url, {cache: 'no-store'});
```

Incremental Static Regeneration

Background revalidation

```
fetch(url, {next: { revalidate: 10} });
```

Revalidar el resultado pasados 10 segundos

Cache NextJS

Los `fetch()` NO se guardan en la cache si:

- `cache: 'no-store'` se agrega al fetch individual
- `revalidate: 0` se agrega al fetch individual
- El `fetch()` se hace dentro de Router Handler (api)
- El `fetch()` se hace luego del uso de headers o cookies
- Se usa el *route segment*

```
export const dynamic = 'force-dynamic'
```

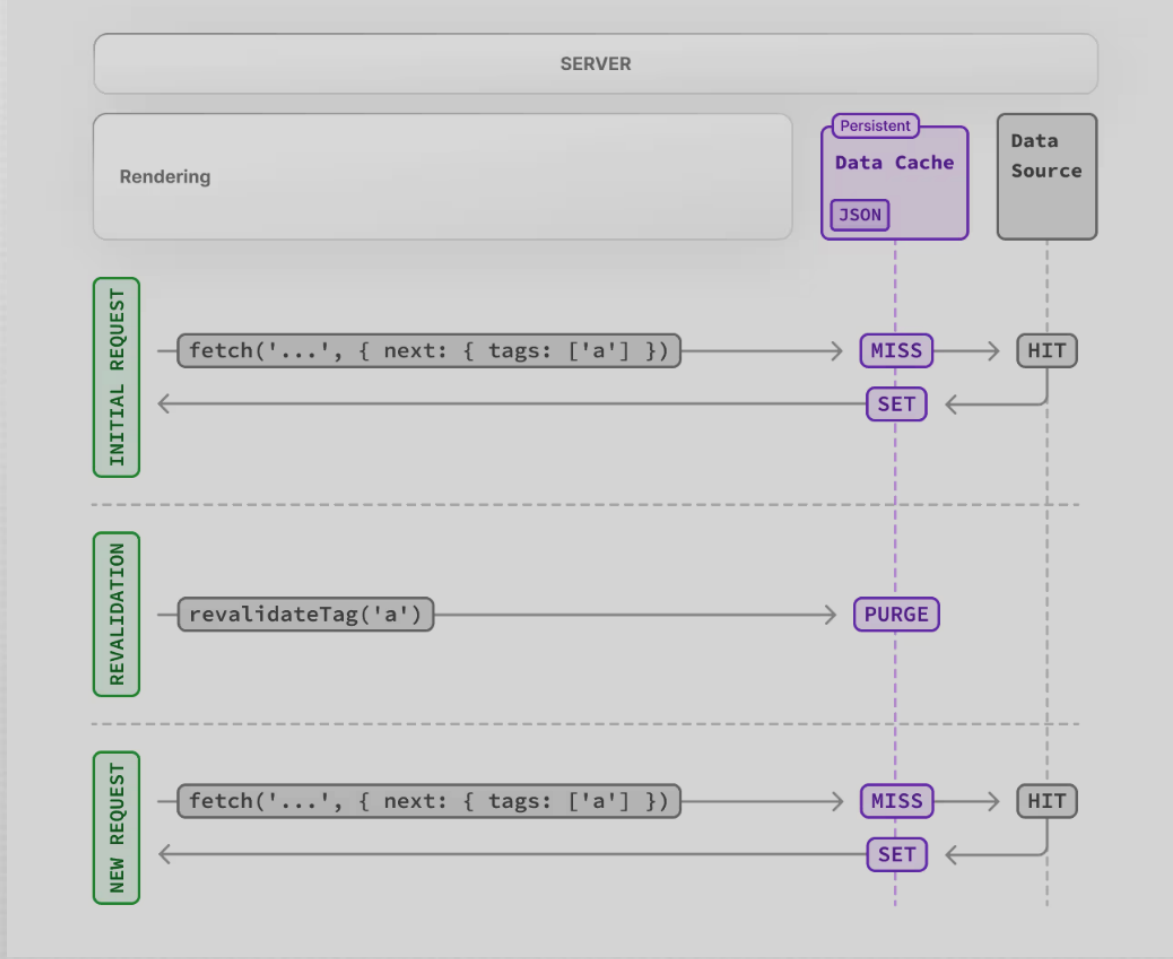
- El *route segment* `fetchCache` se usa indicando saltear la cache (no usar)
- El `fetch()` utiliza datos de autorización y hay un requerimiento no-cacheado arriba en el arbol de componentes.

On-demand revalidation

```
export default async function Page() {
  const res = await fetch('https://...', { next: { tags: ['collection'] } })
  const data = await res.json()
  // ...
}
```

Agrega un tag a los datos obtenidos para identificar la orden de revalidar

On-demand revalidation



On-demand revalidation

Ejemplo de disparador como endpoint API (app/api/route.tsx)

```
import { revalidateTag } from 'next/cache';
import { NextResponse } from 'next/server';

export async function GET(request:NextRequest) {
  const tag = request.nextURL.searchParams.get('tag');
  revalidateTag(tag);
  return NextResponse.json({ message: "Revalidado" },
                           { status: 200 });
}
```

Links

<Link> ≠ <a>

Elemento de Next

Elemento de HTML

Permite navegar sin recargas innecesarias
Soporta pre-fetching
Preferible para rutas y URL locales, no externas
Genera links HTML válidos y accesibles

```
import Link from 'next/link';
...
<Link   href="/pagina/etc"
        prefetch=false
>
        Ver Siguiente
</Link>
```

Imagenes

`<Imagen>` ≠ `<img>`

Elemento de Next

Elemento de HTML

Optimiza imágenes
Evita CLS (Cumulative Layout Shift)
Permite carga diferida (Lazy Load)

```
import Image from 'next/image';
...
<Image
      src="/ruta/a/imagen.jpg"
      alt="Descripción de la imagen"
      width={500}
      height={300}
/>
```

Scripts

`<Script>` ≠ `<script>`

Elemento de Next

Elemento de HTML

Permite cargar scripts externos de manera optimizada y controlada

```
<Script
        src="https://example.com/script.js"
        strategy="beforeInteractive"
/>
```

`beforeInteractive`: Carga antes que cualquier código Next y antes de hidratación
`afterInteractive`: (default) Carga temprano pero puede ocurrir hidratación en la página.
`lazyOnload`: Carga durante el tiempo ocioso en el navegador

Asistencia IA

Algunas consultas interesantes para Web Dev
(si elaboramos mas el prompt, mejor)

- ¿Cómo puedo hacer esta arquitectura más escalable?
- Analiza la calidad de este código
- Analiza este código según su adhesión a los principios SOLID
- ¿Sigue este código las buenas prácticas de accesibilidad?
- ¿Puedes simplificar esta función?
- ¿Hay una forma más limpia y ordenada de controlar estas condiciones?
- ¿Hay algún riesgo de seguridad en este código?

Asistencia IA

Algunas consultas interesantes para Web Dev
(si elaboramos mas el prompt, mejor)

¿Puedes sugerirme un patrón de diseño para esta funcionalidad X?

¿Puedes explicarme el concepto de <*X*> con un ejemplo en <*JavaScript*>?

Sugiere formas de reducir el tiempo de carga en esta aplicación

Sugerir una estructura de carpetas escalable para este proyecto